

Viva: Project 25, Name Predictor

API: Agify + Genderize + Nationalize (<https://agify.io>) - Key: No key needed

Brief: Enter a name and predict age, gender and nationality, three parallel Retrofit calls.

30 questions, roughly easy to hard. Sections A and B are common ground for every project; C and D are specific to this one. Ask 8 to 12 in a typical 10 minute viva.

A. Kotlin and Android basics

1. What is the difference between val and var, and which did you use for your Retrofit instance?
2. What is a data class, and what does Kotlin generate for you?
3. You have three different responses. Did you model three classes or one combined class, and why?
4. What does ? mean on a property, and which prediction fields are genuinely optional?
5. Which Android component holds the three results, and what happens on rotation?
6. How do you represent a screen that is partly loaded, where one of three calls has returned?
7. Where do you declare the INTERNET permission, and what happens if you forget it?

B. Retrofit, coroutines and networking

8. All three APIs share the same shape but different hosts. How did you configure Retrofit?
9. Did you build three Retrofit instances or one with full URLs? Defend your choice.
10. Which annotation carries the name parameter to all three?
11. Why is your API call function marked suspend?
12. This is the parallel calls project. How exactly did you run the three requests concurrently?
13. What is the difference between launch and async, and why does async suit this app?
14. If you awaited each call one after another, how much slower would the screen be?
15. What does coroutineScope do if one of the three calls throws, and is that the behaviour you want?

C. The three APIs and this app

16. What are the three endpoints, and what does each return for the name "michael"?
17. Agify returns age and count. What does count tell you about confidence?
18. Genderize returns gender and probability. How did you present a probability of 0.62 honestly?
19. Nationalize returns a country array of codes with probabilities. How did you display more than one?
20. The country codes are ISO alpha 2. How did you turn "IN" into "India" or a flag?
21. What do all three return for a nonsense name like "xyzyz"? Which fields go null?
22. How does your UI handle a null age or a null gender without crashing?
23. These APIs allow 100 requests per day per IP on the free tier. How does that shape your search behaviour?
24. Which response header tells you how many requests you have left today?

D. Edge cases, design and extensions

25. What does your app show while the three requests are in flight?
26. A user has no connection and searches. Describe exactly what they see.

27. Two calls succeed and one fails. Do you show partial results or an error? Defend your choice.
28. Should you search on every keystroke given the 100 per day limit? What is debouncing?
29. How would you cache results per name so repeat searches cost nothing?
30. These predictions are statistical guesses about real people. What would you add to the UI so a user does not mistake them for facts?

Humble Coders project viva preparation. These are the kinds of questions you should be able to answer about the app you built. Knowing the list is not the same as knowing your own code.